Rukshan De Silva

MSE801_Research_Question

MSE801.1 Research Methods

Document Details

Submission ID
trn:oid:::3618:127746917

Submission Date
Feb 8, 2026, 4:04 PM GMT+13

Download Date
Feb 8, 2026, 4:05 PM GMT+13

File Name
MSE801_Research_Question.pdf

File Size
647.9 KB

23 Pages

5,890 Words

42,760 Characters

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

80 Not Cited or Quoted 13%
Matches with neither in-text citation nor quotation marks

7 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
7% Publications
11% Submitted works (Student Papers)

Match Groups

- **80** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **7** Missing Quotations 1%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
7% Publications
11% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.mdpi.com 2%
2	Student papers	Ravensbourne on 2025-03-17 1%
3	Internet	kipdf.com <1%
4	Internet	staging.preprints.org <1%
5	Student papers	Glyndwr University on 2024-11-13 <1%
6	Internet	ourarchive.otago.ac.nz <1%
7	Student papers	University of Hertfordshire on 2024-11-10 <1%
8	Student papers	Yoobee on 2025-09-10 <1%
9	Student papers	UNICAF on 2025-03-02 <1%
10	Student papers	University of Western Sydney on 2021-06-04 <1%

11 Internet
ejournal.nusamandiri.ac.id <1%

12 Publication
Alex Escalante-Viteri, David Mauricio. "Artificial Intelligence in Software Testing: ... <1%

13 Student papers
Liverpool John Moores University on 2024-03-03 <1%

14 Publication
"Proceedings of the International Conference on AI and Robotics", Springer Scien... <1%

15 Student papers
Amman Arab University for Graduate Studies on 2025-03-01 <1%

16 Student papers
Asia Pacific University College of Technology and Innovation (UCTI) on 2025-09-17 <1%

17 Student papers
Victoria University on 2025-04-11 <1%

18 Internet
archive.conscientiabeam.com <1%

19 Internet
docplayer.info <1%

20 Internet
link.springer.com <1%

21 Student papers
University of Greenwich on 2024-05-08 <1%

22 Internet
edupij.com <1%

23 Student papers
California Southern University on 2026-01-26 <1%

24 Publication
Alfredo Daza, Gonzalo Apaza-Perez, Katherine Samanez-Torres, Juan Benites-Nori... <1%

39 Publication

Bahman Arasteh, Asgarali Bouyer, Peri Gunes, Reza Ghanbarzadeh, Farhad Solei... <1%

40 Student papers

Berlin School of Business and Innovation on 2025-06-16 <1%

41 Student papers

CSU Northridge on 2025-05-12 <1%

42 Student papers

CTI Education Group on 2025-11-24 <1%

43 Student papers

Open University on 2025-10-15 <1%

44 Student papers

Royal Holloway and Bedford New College on 2023-07-03 <1%

45 Student papers

Royal Holloway and Bedford New College on 2023-08-29 <1%

46 Student papers

University of Hertfordshire on 2025-08-06 <1%

47 Internet

www.medischcontact.nl <1%

48 Student papers

Liverpool John Moores University on 2023-02-04 <1%

49 Student papers

North Shore International Academy on 2024-07-07 <1%

50 Publication

Renliang Wang, Feng Liu. "A Software Defect Prediction Method Enhanced by Extr... <1%

51 Publication

Waleed Albattah, Musaad Alzahrani. "Software Defect Prediction Based on Machi... <1%

COLLEGE OF CREATIVE INNOVATION

MSE801 – Research Methods

Assessment 1: Research Proposal

Research Topic

Effectiveness of AI-Based Tools in Software Defect Prediction

---

Selected Research Article

Albattah, W., & Alzahrani, M. (2024). Software defect prediction based on machine learning and deep learning techniques: An empirical approach. Software Engineering Research, 5(4), 86–102. https://doi.org/10.3390/ser5040086

Student Details

Student Name: Wijemuni Rukshan Madushanka De Silva
Student ID: 270802602
Programme: Master of Software Engineering
Course Code: MSE801 – Research Methods

Lecturer / Supervisor

Lecturer: Saveeta Bai

Submission Date

08/02/2026

Abstract

Software defects have always been a problem in software engineering, and they have resulted in various issues, including system failure, increased maintenance costs, and reduced reliability. Software engineering has, therefore, experienced an increase in the use of artificial intelligence in predicting software defects, especially in recent years, due to the increasing complexity of software systems. The main objective of this paper is to evaluate the effectiveness of artificial intelligence in predicting software defects. The paper will achieve this objective by undertaking a literature review that will strictly adhere to the PRISMA model, which emphasizes the collection and evaluation of various empirical studies that have been published in recent times. The literature review will, therefore, seek insights from various recent research papers on the topic in order to evaluate the effectiveness of artificial intelligence in predicting software defects. The literature review will, therefore, continue to emphasize the effectiveness of artificial intelligence in predicting software defects in all its findings, and this will ensure that a comprehensive overview of the potential benefits that can be obtained from using artificial intelligence in predicting software defects is obtained. By undertaking a comprehensive evaluation of the various empirical studies, the literature review will seek to find out the various approaches that are used by artificial intelligence in improving the quality of software systems in predicting software defects.

Table of Contents

MSE801 – Research Methods *1*

Assessment 1: Research Proposal **1**

Research Topic 1
Selected Research Article 1
Student Details 1
Lecturer / Supervisor 1
Submission Date 1

30 *Abstract* *2*

Table of Figures *5*

1. Introduction *6*

2. Journal Selection (Task 1) *6*

Task 1: PRISMA Diagram with Inclusion and Exclusion Criteria **6**

PRISMA Flow Diagram 7
PRISMA Flow Explanation 8
Inclusion Criteria 8
Exclusion Criteria 9

2.1 Selected Article **9**

2.2 Why This Research Paper Was Selected **9**

3. Reading and Analysis (Task 2) *9*

3.1 Title of the Paper **9**

3.2 Authors **9**

27 **3.3 Publication Details** **10**

3.4 Research Topic **10**

3.5 Methodology **10**

4. Research Gaps (Task 3) *10*

Gap 1: Limited Use of Industrial Datasets **10**

Gap 2: Class Imbalance in Defect Data **11**

Gap 3: Lack of Explainable AI Models **11**

8 *5. Research Questions (Task 4)* *11*

RQ1: How effective are AI-based tools in predicting software defects across different datasets and software contexts? 11

RQ3: What research gaps must be addressed to improve the adoption of AI-based software defect prediction tools in industry? 11

Summary of Research Question Scope 12

Research Scope and Conceptual Flow Diagram 12

Diagram Explanation (What the diagram shows) 13

6. Justification of Research Questions (Task 5) 13

6.1 Justification for RQ1: Evaluating Effectiveness Across Contexts 13

6.3 Justification for RQ3: Identifying Research Gaps and Future Improvement Areas 14

6.5 Summary of Why These Questions Strengthen the Research 14

7. Research Hypotheses 15

Hypothesis–Objective Mapping Table 15

Summary of Hypothesis Rationale 16

8. Theoretical Framework 16

9. Research Methodology 17

9.1 Linkage Between Research Gaps, Research Questions, Hypotheses, and Methodology 17

Integrated Research Linkage Section 17

Linkage Between Research Gaps and Research Questions 17

Linkage Between Hypotheses and Research Methodology 18

9.2 Case Studies 18

Case Study 1: Open-Source Software Projects 18

Case Study 2: Enterprise Software Systems 18

Case Study 3: Ensemble Learning Approaches 18

9.3 Research Approach: Mixed Methods 18

10. Data Collection Process 19

11. Strengths and Weaknesses 19

11.1 Strengths 19

Strength 1: Use of a Systematic Literature Review Methodology 19

Strength 2: Focus on Recent and Relevant Research 19

Strength 3: Domain-Specific Focus on Software Engineering 19

11.2 Weaknesses 19

Weakness 1: Dependence on Secondary Data Sources 19

Weakness 2: Limited Availability of Industrial Datasets 20

Weakness 3: Variability in Evaluation Metrics 20

12. Research Completion Timeline 20

Phase 1: Research Planning and Protocol Development (Weeks 1-2) 20

Phase 3: Screening and Duplicate Removal (Week 5) 20

Phase 4: Full-Text Review and Quality Assessment (Weeks 6-8) **20**

Phase 5: Data Extraction and Synthesis (Weeks 9-10) **21**

Phase 6: Analysis and Interpretation (Weeks 11-12) **21**

Phase 7: Report Writing and Finalization (Weeks 13-14) **21**

Summary of Timeline **21**

13. Conclusion ***22***

14. References ***23***

Table of Figures

Figure 1. Prisma Flow Diagram 7

Figure 2. Conceptual diagram illustrating the scope of research questions and their relationship to research gaps, hypotheses, and methodology 12

Figure 3. Theoretical framework illustrating the relationship between software metrics 17

Figure 4. Gantt chart illustrating the timeline for the research methods project. 21

Table 1. Summary of Hypothesis Rationale 16

1. Introduction

Software quality is an important factor for the success of modern software systems. In fact, defects in software systems may cause failure of the systems, as well as increased maintenance and financial costs. In safety-critical domains like healthcare, finance, and transportation, defects in software systems may cause critical problems. Therefore, early detection of defects in software systems is an important objective of modern software engineering practice.

Though traditional approaches like manual testing, code inspections, and static analysis for defect detection in software systems have been widely used in software systems, these approaches suffer **from** some drawbacks in modern software engineering practice. In fact, these approaches may take considerable time and require expertise from human analysts. In modern software systems, with **the** increased complexity of software systems, these approaches may not be effective in analyzing large amounts of historical data for defect detection. This has forced researchers and practitioners of modern software engineering practice to use artificial intelligence and machine learning-based approaches for defect detection in software systems.

The use of artificial intelligence-based software defect detection tools has the potential for effective allocation of testing resources and quality assurance of software systems. Though the use of artificial intelligence-based software defect detection tools has attracted the attention of researchers in modern software engineering practice, there exist some drawbacks in evaluating the effectiveness of these tools using existing research in software engineering practice. Therefore, the objective of this research is to evaluate the effectiveness of artificial intelligence-based software defect detection tools using **a** systematic review of existing research in software engineering practice.

2. Journal Selection (Task 1)

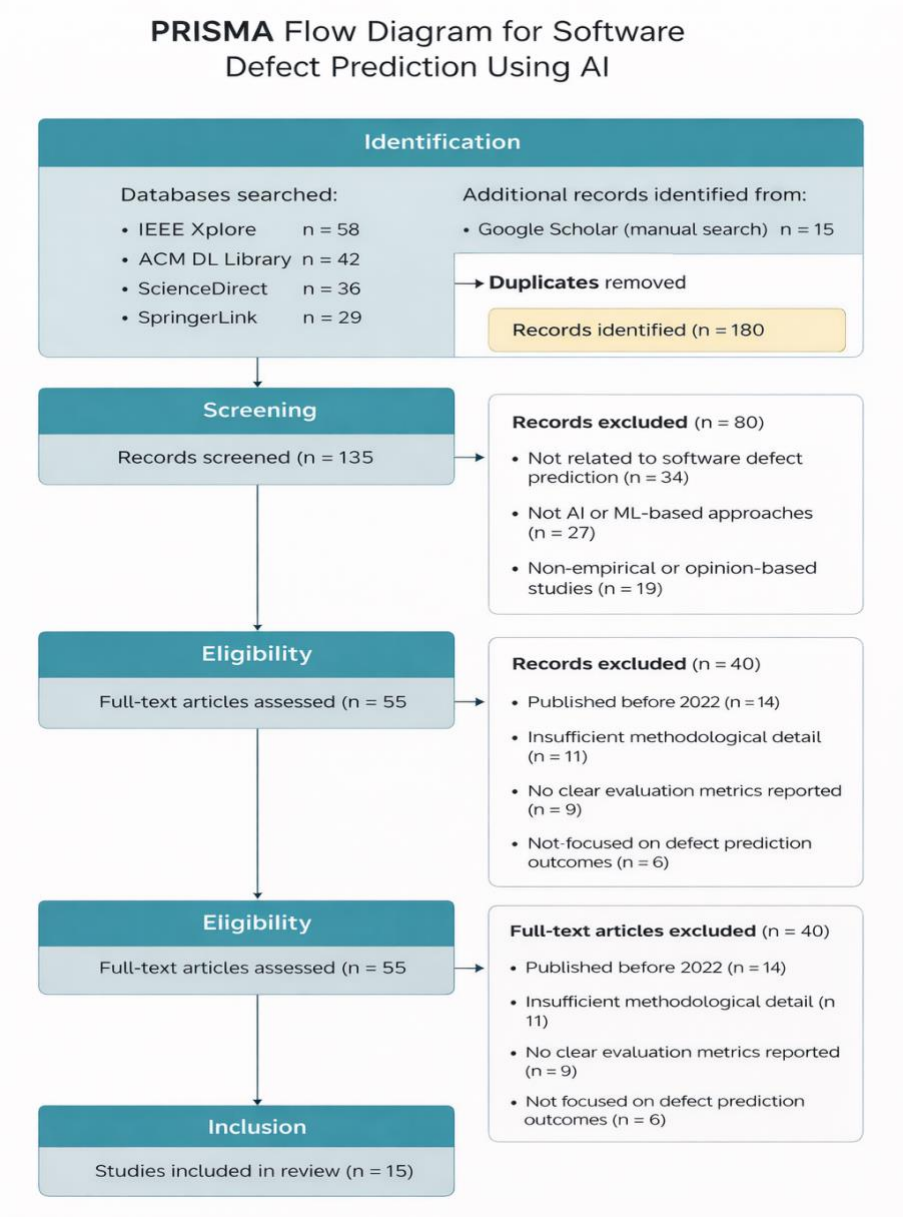
Task 1: PRISMA Diagram with Inclusion and Exclusion Criteria

This research will be conducted in a systematic manner in accordance with the PRISMA framework in identifying, evaluating, and selecting relevant studies that can be included in assessing the effectiveness of AI tools in defect prediction in software engineering. The PRISMA framework was adopted in this research because of its transparency and systematic approach in conducting a systematic review, which is essential in maintaining academic rigor in research without any bias in software engineering research.

The PRISMA framework offers a guideline in documenting the decision-making process in each step in the review. This enhances its credibility, allowing anyone to replicate the research review and evaluation process.

PRISMA Flow Diagram

18 Figure 1 presents the PRISMA flow diagram illustrating the study selection process used in this systematic literature review.



1 **Figure 1. Prisma Flow Diagram**

PRISMA Flow Explanation

In the identification phase, records were collected from various academic databases, such as IEEE Xplore (58 records), ACM Digital Library (42 records), ScienceDirect (36 records), and SpringerLink (29 records). In all, a total of 165 records were collected from these academic databases. To obtain more records, the researchers also performed a manual search using Google Scholar, increasing the number of records to 180.

In the screening phase, duplicate studies (n=45) were eliminated to ensure the accuracy of the results. The remaining 135 studies were subjected to a title and abstract screen to check their relevance to the topic of AI-based software defects. In this phase, a total of 80 studies were eliminated due to reasons such as non-relevance, lack of use of AI or ML, and non-empirical focus.

The eligibility phase consisted of the evaluation of 55 studies based on their full text. The studies were then subjected to inclusion and exclusion criteria to assess their quality. Out of the 55 studies, 40 were excluded for reasons such as being published prior to 2022, lack of adequate methodology, and the absence of evaluation metrics.

Finally, during the inclusion phase, there were 15 studies that met all the eligibility criteria, which were included in the final systematic review. This formed the basis of analyzing the current AI-based approaches used in software defect prediction.

Inclusion Criteria

The inclusion criteria were defined to ensure that selected studies were relevant, current, and methodologically sound:

- Peer-reviewed journal articles
- Publications between 2022 and 2025
- Focus on artificial intelligence, machine learning, or deep learning approaches for software defect prediction
- Empirical studies with clearly reported methodologies and evaluation metrics
- Articles written in English

Exclusion Criteria

Studies were excluded based on the following criteria:

- Publications released before 2022
- Opinion-based papers, editorials, or tutorial articles
- Research not directly related to software defect prediction
- Studies lacking sufficient methodological detail or evaluation results
- Duplicate publications

turnitin Page 14 of 29 - Integrity Submission Submission ID trn:oid:::3618:127746917

2.1 Selected Article

The primary article selected for detailed analysis in this study is:

Software Defect Prediction Based on Machine Learning and Deep Learning Techniques: An Empirical Approach. (Albattah & Alzahrani, 2024)

Link to the Article - Software defect prediction based on machine learning and deep learning techniques: An empirical approach

The study by Albattah and Alzahrani (2024) investigates the application of machine learning and deep learning techniques, including ensemble-based models, for software defect prediction using historical software metrics derived from publicly available datasets.

2.2 Why This Research Paper Was Selected

This paper was selected because it serves well to complement the principal goal of this research, which is to evaluate the efficiency of AI-related tools in defect prediction. This paper provides a thorough empirical study of a few machine learning techniques, as well as ensembles, making it possible to compare them for better prediction results.

The evaluation parameters used are standard, i.e., accuracy, precision, recall, and F1-score. This ensures that the methods employed are in tune with standard practices followed in software engineering research studies. Another advantage is that the paper has used multiple data sets, which adds to the reliability of the study. Finally, the paper is quite recent, which ensures that the developments in AI and ML have been captured well. This makes the paper a good read.

3. Reading and Analysis (Task 2)

3.1 Title of the Paper

Software defect prediction based on machine learning and deep learning techniques: An empirical approach.

3.2 Authors

The paper is written by W. Albattah and M. Alzahrani, and both authors are known for their prominent contributions in software engineering and artificial intelligence fields and studies. The authors are focused on machine learning and deep learning solutions for solving software quality-related issues, particularly predictive analytics and defect predictions.

The researchers have conducted thorough empirical research and provided important and relevant insights into the utilization and application of artificial intelligence for improving software quality, as well as its limitations in software development environments.

3.3 Publication Details

In fact, this particular report was published in the academic journal known as Software Engineering Research. This particular journal was published in the year 2024 and allows the readers to get access to information related to software quality, AI applications, and software engineering techniques, among others. Additionally, it has been established that software engineering is related to software applications.

3.4 Research Topic

The article researched the role that machine learning and deep learning techniques play in the prediction of software defects. The primary aim of the study is to enhance detection at an early stage based on past software metrics or development information using artificial intelligence-based prediction models. Ultimately, this will ensure that parts of the code that are suspected of having defects are identified early to inform the development of a much-improved software testing strategy or quality assurance process.

3.5 Methodology

In the study, supervised learning techniques are applied by experimenting with different classification algorithms to predict whether a software module is defect-prone or not. For this purpose, publicly available datasets are employed to evaluate the performance using metrics such as accuracy, precision, recall, and F-measure.

From the results, it is observed that ensemble methods outperform most individual models, thereby highlighting the benefits of using a combination of classifiers. However, as suggested by the work, difficulties with data, class imbalance, and poor generalization to real-world industrial software also remain.

Additionally, this research employs a form of mixed approach that integrates a literature review and case study analysis qualitatively.

4. Research Gaps (Task 3)

Gap 1: Limited Use of Industrial Datasets

One of the main gaps in the existing research was the significant reliance of all the existing research on open-source data sets. These data sets are excellent for experimentation; however, in real-world scenarios facing the complexity of the actual systems in the real world, they can only go so far in providing the practical effectiveness of the AI-driven models of defect prediction.

Gap 2: Class Imbalance in Defect Data

Most of the defect prediction datasets suffer from strong class imbalance; there are far more non-defective instances than defective ones.

This skew can bias machine learning models and lead to misleading performance measures with weaker reliability, especially for safety-critical systems.

Gap 3: Lack of Explainable AI Models

While AI-based defect prediction models may yield high prediction accuracy, these models may not be easily interpretable, and hence, they may be considered as black box systems, dissuading software professionals in industries due to their lack of transparency.

5. Research Questions (Task 4)

The study proposes a series of research questions, which in essence seek a deliberate and systematic exploration of how well these tools perform in terms of defect prediction. Each of these research questions is also concerned with different aspects of this problem. They present the scope, boundaries, and intent of the study itself. (Li, Jing, & Zhu, 2018)

RQ1: How effective are AI-based tools in predicting software defects across different datasets and software contexts?

Scope of RQ1:

This question has to do with the actual performance of AI-powered defect prediction tools, as illustrated in recent research. It weighs their effectiveness across various datasets, project kinds, and testing environments. Major metrics to be considered include accuracy, precision, recall, and F1-score, comparing several machine learning models and ensemble approaches. The objective is to synthesize existing evidence on the extent to which AI tools reliably forecast defects across different software contexts, rather than building new models.

RQ2: What limitations affect the accuracy and practical applicability of current AI-based defect prediction models?

Scope of RQ2:

This paper seeks to examine how effective AI-based defect prediction models could possibly be, looking deeper into issues like data quality, class imbalance problems, overfitting models, and issues of evaluation bias, as well as the realities of making models that become comprehensible, making such models cost-effective, and understanding whether software development processes incorporate such models. It will also look deeper into what really matters when it comes to making use of such models by software engineering teams.

RQ3: What research gaps must be addressed to improve the adoption of AI-based software defect prediction tools in industry?

Scope of RQ3:
It points out what is missing in current research that slows down the use of AI-driven defect prediction tools in actual industrial environments. It also points to the few industrial datasets, limited real-world settings validation, the absence of explainable AI methods, and the absence of standardized results evaluation. It critically aims to drive further research toward those missed gaps for both scholarly understanding and practical progress in software engineering.

Summary of Research Question Scope
These questions clearly define a specific scope of work that can be achieved with the research. These questions guarantee that the paper will evaluate the effectiveness (RQ1), limitations (RQ2), and possibilities for improvement in the next steps (RQ3). In doing so, the research becomes more cohesive and relevant while maintaining its integrity in evaluating AI-powered defect prediction tools. (Hall, Beecham, Bowes, Gray, & Counsell, 2012)

Research Scope and Conceptual Flow Diagram

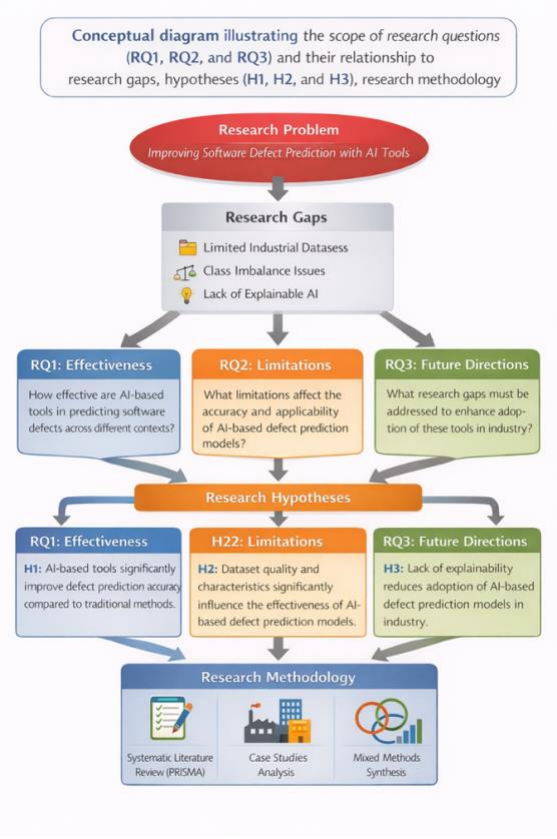


Figure 2. Conceptual diagram illustrating the scope of research questions and their relationship to research gaps, hypotheses, and methodology

Diagram Explanation (What the diagram shows)
The scope diagram outlines how the questions of the study shape its structure and tie into what's missing in the field, the hypotheses, and the chosen methods. It starts with the objective of enhancing software defect prediction using AI tools, based on the shortcomings within the literature review-things like small datasets and a shortage of explainable AI models.

From that basis, three core questions follow: the effectiveness question RQ1, the limitations question RQ2, and finally, the improvement potential in the future question RQ3. The questions thus propagate into hypothesis development, to be tested through a systematic review of literature and multiple case studies. The diagram also describes the adopted methods, PRISMA-guided reviews, and the approach for mixed-methods synthesis, explaining problem identification in relation to research design and analysis. Basically, this visual enables the reader to make sense of the logical flow and the scope of this study.

6. Justification of Research Questions (Task 5)

The questions that were developed by the study were developed to specifically direct a systematic evaluation of how well these kinds of AI tools perform for defect prediction within a program of work or software. Each question addresses a different dimension: how well they perform, what realities constrain usage, and what needs to improve going forward. They nonetheless remain descriptive and evaluative with clear and actionable implications for software engineering.

6.1 Justification for RQ1: Evaluating Effectiveness Across Contexts

RQ1: How effective are AI-based tools in predicting software defects across different datasets and software contexts?

Thus, this question is significant as it cannot simply be determined based on a set of data or a particular project type, as it is concerning AI models, which may or may not perform well based on different project domains, project sizes, and software development methodologies. RQ1 is concerned with whether AI-based defect prediction tools can actually work well or remain conditioned for only specific criteria. The results of defect prediction can play a crucial role in deciding which modules of software need to be tested first, etc. If defect prediction is not effective for all cases, it can ultimately increase defects.

6.2 Justification for RQ2: Understanding Limitations and Barriers to Practical Use

RQ2: What limitations affect the accuracy and practical applicability of current AI-based defect prediction models?

While these studies may have demonstrated state-of-the-art accuracy, AI systems appear to hit stumbling blocks as soon as they enter software teams. Needless to say, it is difficult to achieve top-notch accuracy by using figures. RQ2 questions what actual problems AI models face in practical situations, including technical issues like class imbalance, noisy data, overfitting, as well as disconnects between how models were developed and how they are employed. Moreover, it also raises questions about difficulties in computing price, integration with current systems, and, most importantly, transparency to developers. Knowing these limits is relevant in situations where developers need AI models to be interpretable, stable, and simple to incorporate into existing systems. RQ2 asks questions about why AI models fail, not fundamentally which AI model succeeds.

6.3 Justification for RQ3: Identifying Research Gaps and Future Improvement Areas

RQ3: What research gaps must be addressed to improve the adoption of AI-based software defect prediction tools in industry?

RQ3 is important in a methods-focused article because one does not gain ground by rehearsing what is already obvious. There are streams of works on new algorithm proposals, but only a few projects attack the big questions around explainability, data quality, and how we should actually evaluate these tools. RQ3 can elucidate reasons why AI-driven defect prediction tools struggle for adoption in industry, binding the value of a study to its limitations and showing how those limitations could inspire future directions. The fact that a study depends on open-source data today often suggests that the best follow-up work involves industry partners more directly than before. In brief, RQ3 nudges this work beyond an enumeration of what is already known into an agenda of what is to be done.

6.4 Alignment with Research Gaps and Study Objectives

The study ascertains its main focus around three key research queries that aim at filling the gaps discussed earlier. RQ1 seeks to investigate the effectiveness of the proposed approach on multiple datasets, as previously discussed. RQ2 proposes an endogenous measure that can alleviate issues of class imbalance and the need for model interpretability, as earlier discussed. RQ3, as discussed earlier, addresses the absence of data in the real-world industry and the need for interpretable AI, crucial in the advancement of SE research and practice.

6.5 Summary of Why These Questions Strengthen the Research

The questions that are being addressed by the research are based on assessing the effectiveness of AI-based defect prediction from a technical perspective and a software engineering perspective. This adds academic rigor to the research, making it more relevant to software engineering practitioners who are looking for effective and efficient AI tools.

7. Research Hypotheses

The research hypotheses in this study are formulated based on patterns observed in existing literature and the theoretical foundations of software metrics and machine learning. Each hypothesis represents an assumption that can be examined through systematic analysis of prior empirical studies.

H1: AI-based tools significantly improve software defect prediction accuracy compared to traditional defect detection methods.

Reasoning:

This hypothesis assumes that AI tools can learn complex patterns from historical software data better than traditional methods.

Machine learning can analyze large amounts of software metrics, find non-linear relationships, and adapt to changing data. Previous studies show that AI models often have higher prediction accuracy than manual inspections and static analysis. H1 expects measurable benefits in defect prediction from AI approaches and seeks to verify this with empirical evidence.

H2: Dataset quality and characteristics significantly influence the effectiveness of AI-based software defect prediction models.

Reasoning:

AI-based defect prediction models depend significantly on the quality of training data. Problems like noisy data, missing values, and class imbalance can harm model performance. Studies show that models trained on clean, balanced datasets perform better. This suggests that predictive performance depends on dataset characteristics as well as algorithm choice. Examining H2 helps determine if limitations are technical or data-driven.

H3: The lack of explainability in AI-based defect prediction models reduces their practical adoption in industrial software development environments.

Reasoning:

Many AI models are accurate, but their black-box nature can lead to distrust among software developers and quality assurance engineers. They need clear explanations for predictions to make informed choices, especially in safety-critical areas. The study will explore if lack of transparency is a consistent barrier to using AI-based defect prediction tools in practice.

Hypothesis–Objective Mapping Table

Table 1. Summary of Hypothesis Rationale

Hypothesis	Research Objective	Explanation of Alignment
H1: AI-based tools significantly improve software defect prediction accuracy compared to traditional methods.	To evaluate whether AI-based defect prediction models outperform traditional defect detection approaches.	This hypothesis aligns with the objective of assessing effectiveness by comparing reported performance metrics across AI-based and traditional approaches in the literature.
H2: Dataset quality and characteristics significantly influence the effectiveness of AI-based defect prediction models.	To analyze how dataset properties such as size, balance, and noise affect prediction accuracy and generalizability.	This mapping ensures that performance is evaluated not only based on algorithms but also on the quality and representativeness of datasets used in prior studies.
H3: Lack of explainability reduces adoption of AI-based defect prediction models in industry.	To identify factors that limit practical adoption of AI-based tools in real-world software development environments.	This hypothesis connects technical performance with usability and trust, addressing the gap between academic research and industrial application.

This table strengthens the methodological clarity of the study by explicitly linking hypotheses to measurable research objectives.

Summary of Hypothesis Rationale

Together, these hypotheses support a balanced evaluation of AI-based defect prediction tools by addressing performance improvement (H1), data dependency (H2), and practical usability (H3). Testing these assumptions through a systematic review strengthens the analytical depth of the study and ensures alignment between theoretical expectations and empirical evidence.

8. Theoretical Framework

This study is grounded in three complementary theoretical perspectives: software metrics theory, machine learning theory, and defect prediction theory.

Software metrics theory explains how measurable attributes such as code complexity, coupling, cohesion, and size are associated with defect occurrence. These metrics provide meaningful inputs for AI-based prediction models.

Machine learning theory provides the foundation for predictive modelling by enabling systems to learn patterns from historical data through supervised learning techniques.

Defect prediction theory suggests that historical defect data can be used to forecast future defect-prone components. Together, these theories explain how AI-based tools support early defect detection.

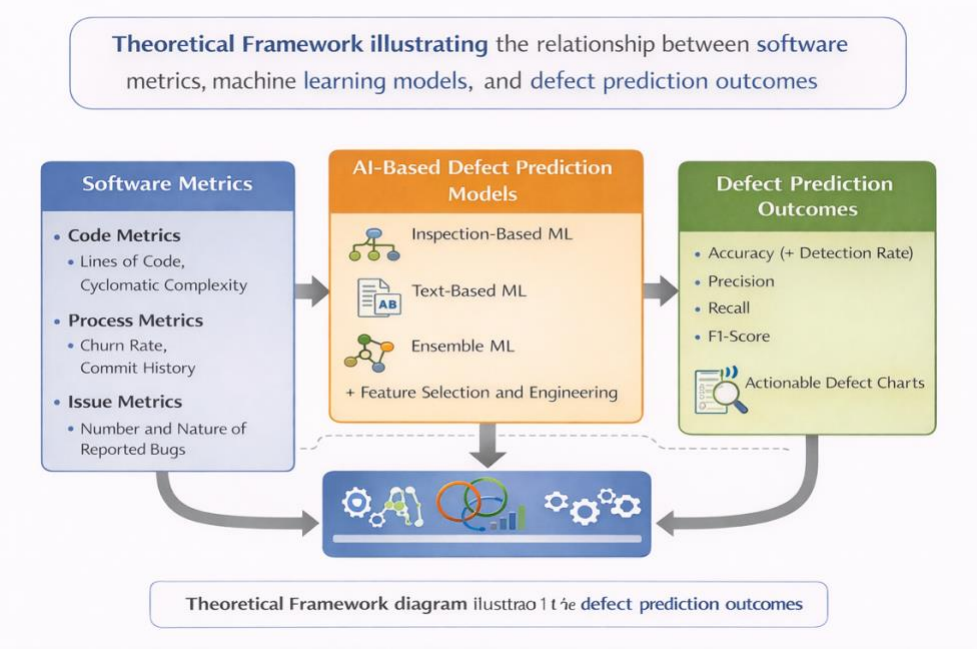


Figure 3. Theoretical framework illustrating the relationship between software metrics

This diagram visually represents how software metrics are processed by machine learning models to produce defect prediction outcomes. (Malhotra, 2015)

9. Research Methodology

9.1 Linkage Between Research Gaps, Research Questions, Hypotheses, and Methodology

Integrated Research Linkage Section

This section explains how the key components of the research—identified gaps, research questions, hypotheses, and methodology—are systematically connected to ensure coherence and academic rigor.

Linkage Between Research Gaps and Research Questions

The research gaps found in this study help shape the research questions. RQ1 looks at effectiveness across different datasets due to reliance on open-source data. RQ2 addresses class imbalance and explainable AI issues, while RQ3 focuses on gaps that affect industrial use.

Linkage Between Research Questions and Hypotheses

Each research question is supported by one or more hypotheses that translate broad investigative questions into testable assumptions. RQ1 aligns with H1 by examining whether AI-based tools truly outperform traditional methods.

RQ2 aligns with H2 by focusing on data-related and technical factors influencing performance. RQ3 aligns with H3 by addressing adoption barriers linked to explainability and trust.

Linkage Between Hypotheses and Research Methodology

The research methodology was chosen to systematically examine each hypothesis. A PRISMA-guided systematic literature review evaluates performance results and limitations, while case studies offer insight into AI-based defect prediction tools. A mixed-methods approach supports both qualitative and quantitative analysis.

Overall Research Coherence

Together, these linkages ensure that the research is logically structured and methodologically sound. The progression from research gaps to questions, hypotheses, and methodology demonstrates clear alignment between the problem statement and the analytical approach. This integrated structure strengthens the validity of the study and ensures that conclusions are grounded in systematic and transparent analysis.

9.2 Case Studies

Several case studies from existing literature were analyzed to understand how AI-based defect prediction tools have been applied in different software development contexts.

Case Study 1: Open-Source Software Projects

AI-based models were applied to large open-source projects to prioritize defect-prone modules. While results demonstrated improved efficiency, challenges related to data noise and inconsistent documentation were identified.

Case Study 2: Enterprise Software Systems

Enterprise-level systems showed improved prediction accuracy due to cleaner datasets, but challenges related to data access, privacy, and organizational resistance were observed.

Case Study 3: Ensemble Learning Approaches

Ensemble techniques improved robustness and accuracy but introduced concerns related to model interpretability and computational complexity.

9.3 Research Approach: Mixed Methods

A mixed-methods approach is adopted, combining qualitative synthesis of published research with quantitative analysis of reported performance metrics. This approach enables a comprehensive evaluation of AI-based defect prediction tools.

10. Data Collection Process

Data for this study were collected from peer-reviewed journal articles identified through systematic searches of major academic databases. A structured data extraction process was used to collect information on datasets, AI techniques, evaluation metrics, and reported outcomes. This ensured consistency, transparency, and reproducibility.

The collected data will be used to compare reported AI-based defect prediction approaches, analyze methodological trends, and identify limitations and research gaps relevant to industrial adoption.

11. Strengths and Weaknesses

11.1 Strengths

Strength 1: Use of a Systematic Literature Review Methodology

A major strength of this study is the use of a systematic literature review guided by the PRISMA framework. This approach ensures transparency, methodological rigour, and reproducibility throughout the research process. By following a structured review methodology, the study minimizes selection bias and enhances the credibility of its findings.

Strength 2: Focus on Recent and Relevant Research

By limiting the review to studies published between 2022 and 2025, the research ensures that findings reflect current advancements in artificial intelligence and software engineering. This focus increases the relevance of the study and ensures that conclusions are applicable to modern software development practices.

Strength 3: Domain-Specific Focus on Software Engineering

The exclusive focus on software defect prediction ensures that findings are directly relevant to software engineering practice. This domain-specific approach avoids generalization across unrelated fields and strengthens the applicability of results to real-world software quality assurance processes.

11.2 Weaknesses

Weakness 1: Dependence on Secondary Data Sources

A key limitation of this study is its reliance on secondary data obtained from published research. As a result, the study has limited control over dataset quality, experimental design, and evaluation methods used in the original studies. This dependence may introduce variability in reported results.

Weakness 2: Limited Availability of Industrial Datasets
Many reviewed studies rely on open-source datasets, which may not accurately represent industrial software systems. The lack of industrial datasets limits the generalizability of findings and reduces the practical relevance of some AI-based defect prediction models.

Weakness 3: Variability in Evaluation Metrics
Another limitation is the variation in evaluation metrics used across studies. Differences in performance measures make direct comparison of results challenging and may affect the interpretation of model effectiveness.

12. Research Completion Timeline

The systematic literature review process was conducted across a structured timeline to ensure thorough and methodical analysis of AI-based tools for software defect prediction. The timeline below outlines the key phases and activities undertaken throughout the research process, from initial planning through to final synthesis and reporting.

Phase 1: Research Planning and Protocol Development (Weeks 1-2)
During this initial phase, the research question was created, and the PRISMA protocol was set. Key activities included defining the review scope, establishing criteria for inclusion and exclusion, identifying relevant databases, and developing a search strategy with keywords like "AI software defect prediction. " Ethical approval was also obtained.

Phase 2: Literature Search and Identification (Weeks 3-4)

Systematic searches in selected databases found 165 records, and a manual search on Google Scholar added 15, totaling 180 studies. All records were stored in a reference management system, ensuring transparency and comprehensive literature coverage from 2022 to 2025.

Phase 3: Screening and Duplicate Removal (Week 5)
Duplicate records (n=45) were removed using tools and manual checks. The remaining 135 studies were screened for relevance, excluding 80. This left 55 studies for full-text review, with checks for consistency.

Phase 4: Full-Text Review and Quality Assessment (Weeks 6-8)
Fifty-five full-text articles were reviewed based on inclusion and exclusion criteria. After evaluating quality and details, 40 studies were excluded, leaving 15 high-quality studies for final inclusion, with reasons for exclusion documented.

Phase 5: Data Extraction and Synthesis (Weeks 9-10)

Data from 15 studies were extracted using a structured form, focusing on study details, AI/ML techniques, datasets, evaluation metrics, performance results, and key findings. The data were synthesized qualitatively and quantitatively, revealing patterns, trends, and gaps in the literature.

Phase 6: Analysis and Interpretation (Weeks 11-12)

The data were analyzed to answer research questions using theories related to software metrics, machine learning, and defect prediction. AI-based defect prediction tools were evaluated, highlighting strengths, limitations, and opportunities for future research in software engineering.

Phase 7: Report Writing and Finalization (Weeks 13-14)

The systematic review report was completed, including all phases of the research process. A PRISMA flow diagram was added, and all sections, such as introduction, methodology, results, discussion, and conclusion, were written. The document was revised and proofread multiple times for clarity and rigor, with references formatted in APA style, ready for submission.

Summary of Timeline

The systematic literature review spanned 14 weeks, facilitating a thorough execution of the PRISMA protocol. This structured timeline ensured a rigorous and credible analysis of AI-based tools for software defect prediction, contributing to the reliability and validity of the findings.

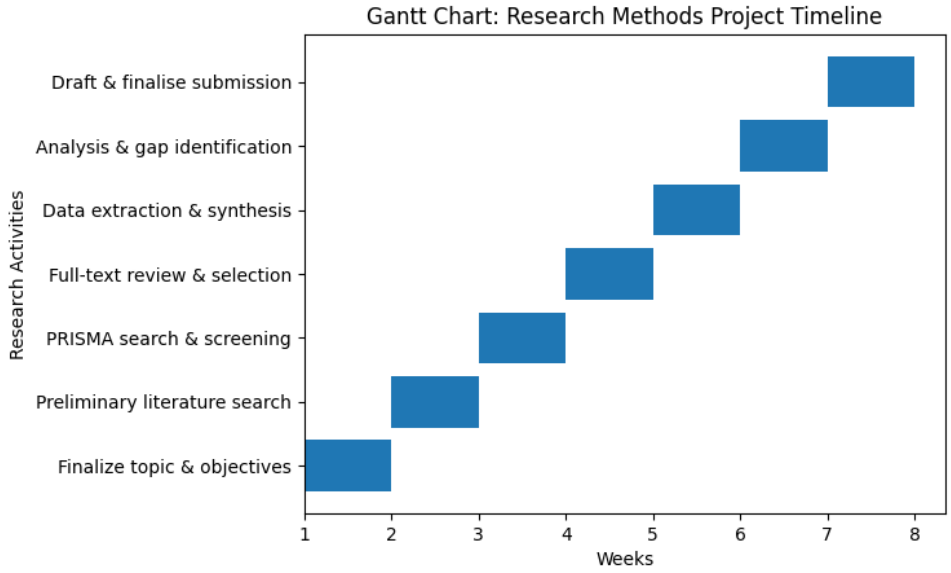


Figure 4. Gantt chart illustrating the timeline for the research methods project.

The Gantt chart gives an overview of the research activities to be conducted, and the period involved in the research process, indicating how the research process will be well organized and achievable to ensure proper time management in the research methods project.

13. Conclusion

In this study, the potential of AI tools in predicting software defects is evaluated by undertaking an in-depth literature review in accordance with the PRISMA protocol. With the increasing complexity of software, existing methods of detecting defects are no longer effective, making it a pressing need to have more intelligent and data-driven solutions. AI and machine learning techniques have shown promise in the early identification of potential software components that are likely to be defective.

The literature review shows that AI tools, especially those that use machine learning and ensemble methods, can improve the accuracy of defect detection and overall software quality. AI tools have shown better performance compared to traditional methods by using historical data metrics to identify hidden patterns of potential software defects.

Despite the promising potential of AI tools in predicting software defects, the literature shows that there are some important gaps that need consideration in order to have AI-based software tools that can effectively predict potential software defects. Among the important gaps in AI tools are that they rely on open-source data, which may not accurately represent the real-world software environment. Class imbalance in data, characterized by a lower proportion of potential software defects, may also affect the overall performance of machine learning techniques. Complexity in AI tools may also limit their overall interpretability.

Despite the gaps in AI tools, they have the potential to improve software quality and efficiency, making way for better AI-based defect prediction tools in software engineering.

11

14. References

Albattah, W., & Alzahrani, M. (2024). Software defect prediction based on machine learning and deep learning techniques: An empirical approach. *Software Engineering Research*, 86–102.

3 Hall, T., Beecham, S., Bowes, D., Gray, D., & Counsell, S. (2012). A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering*, 1276–1304.

6 Li, Z., Jing, X. Y., & Zhu, X. (2018). Progress on approaches to software defect prediction. *IEEE Access*, 6761–6776.

13 Malhotra, R. (2015). A systematic review of machine learning techniques for software fault prediction. *Applied Soft Computing*, 504–518.

4 Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., & al., e. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, n71.